KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 2 ⋄ poznámky k přednášce

1. Lisp pro tento semestr

verze z 13. února 2025

V novém semestru budeme používat jinou, pokročilejší variantu jazyka Lisp, než v semestru minulém. Účelem této přednášky je seznámit vás s většinou nových možností, které poskytuje.

1 Rozdíly proti minulému semestru

Některé možnosti nového jazyka popíšeme v tomto textu, s dalšími se budeme seznamovat průběžně. Jazyk nám umožní některou funkčnost, kterou jsme minulý semestr dostali přímo k dispozici, naprogramovat. Proto ji nejprve nebudeme moci používat. Jde především o operátory `let`, `let*` a `cond`, bez kterých se zatím budeme muset obejít.

Seznam vestavěných speciálních operátorů a funkcí je na konci tohoto textu. Kromě toho *PP Polyglot* při startu vždy načte funkce *do jazyka*, které jsme v minulém semestru programovali na přednáškách a cvičeních. Zdrojový soubor je v aplikaci k dispozici a můžete se na něj podívat.

2 Řetězce

Textový řetězec (*string*) je jednorozměrné pole libovolných znaků. Zapisuje se, jak bývá v programovacích jazycích zvykem: do uvozovek. O obecných jedno- i vícerozměrných polích si řekneme časem. Zatím použijeme jen několik funkcí, které s nimi pracují. Jednorozměrným polím říkáme **vektory**. Textové řetězce jsou tedy vektory.

Z pohledu vyhodnocovacího procesu je řetězec atom, který se vyhodnocuje na sebe podobně jako například čísla nebo symboly `t` a `nil`:

```
> "Dobrý den"
"Dobrý den"
```

Textový řetězec se samozřejmě rovná sám sobě:

```
> (bind string "Dobrou noc")
"Dobrou noc"

> (eql string string)
T
```

Když ale takto zadáme týž řetězec dvakrát, může Lisp pokaždé vytvořit řetězec nový, takže se může stát toto:

```
> (eql "Já" "Já")
NIL
```

Jde totiž o dva různé řezězce, které jen obsahují stejné znaky. S něčím podobným jsme se už setkali u seznamů.

Jak víme, u symbolů k tomu nedochází: když do zdrojového kódu zapíšeme vícekrát stejný symbol, jde opravdu o týž symbol:

```
> (eql 'abc 'abc)
T
```

Znaky v našem Lispu se zadávají syntaxí `#\`. Např. `#\A` je velké písmeno `A`. Některé znaky se dají zadat i názvem, např. `#\space` nebo `#\newline` (možno i s velkými písmeny). To je samozřejmě výhodné zejména pro neviditelné znaky, jako třeba mezeru, kterou ovšem můžeme zadat i takto: `#\ `.

Jelikož textové řetězce jsou jednorozměrná pole znaků, lze k práci s nimi používat obecné funkce pracující s poli. Ke zjištění složky pole daného indexu slouží v Lispu funkce `aref`:

```
> (aref "abc" 1)
#\b

> (aref "ab c" 2)
#\Space
```

Řetězec v následujícím testu obsahuje jako svůj druhý znak (tj. znak s indexem `1`) znak pro nový řádek:

```
> (aref "a
bc" 1)
#\Newline
```

Pokud chceme v textovém řetězci použít uvozovky, použijeme zpětné lomítko. To používáme i pro vkládání zpětného lomítka samotného (toto zatím v Polyglotu nefunguje):

```
> "Uvozovky: \", zpětné lomítko: \\"
"Uvozovky: \", zpětné lomítko: \\"

> (aref * 10)
#\"

> (aref * 11)
#\,
```

Funkce `stra` a `strs` vracejí zápis libovolné hodnoty v textovém řetězci. První funkce hodnotu zapíše bez znaků určujících její typ (např. uvozovek u řetězců), druhá s nimi:

```
> (stra "ahoj")
"ahoj"

> (stra #\a)
"a"

> (strs "ahoj")
"\"ahoj\""

> (strs #\a)
"#\\a"
```

(„a“ znamená *aesthetic*, „s“ *simple*)

Délku textového řetězce (počet znaků) lze zjistit funkcí `array-dimension`:

```
> (array-dimension "abc" 0)
3
```

3 Vedlejší efekt

Až na drobné výjimky, všechny speciální operátory a funkce, o kterých jsme mluvili v minulém semestru, mají jediný účel: pomocí argumentů (ať už vyhodnocených nebo ne, podle toho, o jaký operátor jde), na které byly aplikovány, získat výsledek a ten vrátit jako hodnotu. K žádné jiné změně nedojde: pokud bychom vrácenou hodnotu ignorovali, nemáme jak zjistit, že aplikace proběhla.

To je v souladu s tím, jak funguje vyhodnocovací proces. Během práce programu se **vyhodnocují symbolické výrazy**. Získaná hodnota výrazu se buď použije jako argument pro další výraz (v případě složených symbolických výrazů), nebo je už konečným výsledkem výpočtu. Přitom se nic jiného s programem nestane.

Už jsme se ale setkali s případy, které tomuto čistému pojetí vyhodnocování výrazů nevyhovují. Při následujícím vyhodnocení se stane něco navíc:

```
> (bind a 10)
10
```

Kromě toho, že výraz vrátí hodnotu `10`, ještě nastaví hodnotu proměnné `a` v prostředí Listeneru na `10`. Zatímco před jeho vyhodnocením by pokus vyhodnotit v Listeneru výraz `a` mohl skončit například chybou, nyní chybou neskončí a získáme hodnotu `10`. Takové změně říkáme vedlejší efekt.

K *vedlejšímu efektu* vyhodnocení výrazu dojde v případě, že kromě vrácení hodnoty výraz způsobí nějakou vnější změnu. Tou změnou může být (jako v uvedeném příkladě) změna výchozího prostředí, nebo třeba tiskový nebo jiný výstup.

Prvnímu typu vedlejšího efektu můžeme říkat *interní vedlejší efekt*. Takový efekt způsobil nahoře uvedený výraz, může jím také být mutace hodnoty v datové struktuře nebo třeba definice funkce:

```
(defun exp2 (x)
  (* x x))
```

Před vyhodnocením tohoto výrazu funkce `exp2` nemusela vůbec existovat, po jeho vyhodnocení existuje a dělá to, co je výrazem určeno. Vyhodnocením výrazu jsme tedy vykonali interní vedlejší efekt.

Za *externí vedlejší efekt* považujeme takový, který není možné zpětně programem zjistit. Může to být například tisk nebo grafický výstup na obrazovce. Minulý semestr jsme se s externím vedlejším efektem setkali u funkcionální grafiky. Externí vedlejší efekt je bezpečnější než interní.

Interní vedlejší efekt: nastavení hodnoty proměnné

K nastavení hodnoty proměnné lze použít speciální operátor `setq`:

```
(defun setq-test (a)
  (setq a (1+ a))
  (1+ a))
```

```
> (setq-test 3)
5
```

Nastavovat lze pouze hodnotu proměnných, které mají v aktuálním prostředí vazbu. Změně hodnoty proměnné (a později také složky datové struktury) budeme říkat **mutace** a budeme se jí podrobněji věnovat později.

Externí vedlejší efekt: tisk

Do Listeneru lze *tisknout* libovolnou zprávu funkcí `out`. Funkce přijímá libovolný počet argumentů a všechny vytiskne za sebe do Listeneru bez znaků uvozujících jejich typ, tedy jako by použila funkci `stra` (*estetickým způsobem*). Pokud chceme uvozující znaky vytisknout, použijeme funkci `strs`:

```
> (out "Řetězec " (strs "ahoj") " a znak " (strs #\a) #\.)
Řetězec "ahoj" a znak #\a.
NIL
```

(Polyglot zatím tak hezky netiskne, ale brzy bude.) Funkce vždy vrací `NIL`.

4 Bloky

Při tisknutí (a obecně při použití vedlejšího efektu) může být třeba vyhodnotit několik výrazů za sebou (tzv. sekvenčně). K tomu slouží tzv. *bloky* kódu. Tělo funkcí i některých operátorů mohou být takovým blokem:

```
(defun printing-sqr (x)
  (labels ((res (sqr)
             (out "Vstup funkce printing-sqr: " (strs x)
                  ", výstup: " (strs sqr))
             sqr))
    (res (* x x))))
```

Funkce `printing-sqr` vrací druhou mocninu svého argumentu, takže můžeme udělat například toto:

```
> (sqrt (+ (printing-sqr 3) (printing-sqr 4)))
Vstup funkce printing-sqr: 3, výstup: 9
Vstup funkce printing-sqr: 4, výstup: 16
5.0
```

Tato změna vyžaduje změnit definici těla funkce a dalších operátorů: není to jeden výraz, jak jsme tělo chápali v minulém semestru, ale **seznam výrazů**.

Při práci s vedlejším efektem se někdy hodí vytvářet blok explicitně. K tomu slouží speciální operátor `progn`. Ten akceptuje libovolný nenulový počet argumentů. Při vyhodnocení výrazu

```
(progn expr1 expr2 ... exprn)
```

se postupně (zleva doprava) vyhodnotí všechny výrazy *expr1*, *expr2*, ..., *exprn*, hodnota posledního (tedy *exprn*) se vrátí jako výsledek. Návratové hodnoty předchozích výrazů se ignorují.

Poznámka. Tělem uvedeného výrazu je seznam `(expr1 expr2 ... exprn)`. Pokud ho označíme *body*, můžeme výraz napsat i takto: `(progn . body)`.

Ukázka funkce, která vytiskne všechna nezáporná celá čísla menší než zadané číslo a nakonec vrátí `nil`. Funkce používá speciální operátor `progn`.

```
(defun print-numbers (n)
  (if (<= n 0)
      nil
    (progn
      (out (- n 1))
      (print-numbers (- n 1)))))
```

Test:

```
> (print-numbers 10)
9
8
7
6
5
4
3
2
1
0
NIL
```

K podobným účelům by se ale více než rekurze hodily cykly.

5 Chyby

Běžící program se nachází ve **výjimečném stavu**, když nejsou splněny předpoklady pro jeho další bezchybnou funkci. Obvykle jde o předpoklady o hodnotách proměnných. Pokud například vytvoříme vazbu symbolu na hodnotu `0`, předpokládáme, že v daném prostředí ji opravdu bude mít. Pokud nepočítáme s vedlejším efektem, bude tento předpoklad porušen jedině v důsledku hardwarové chyby (chyba paměti) nebo závažné chyby softwarové (nahodilé přepsání bloku paměti).

Hlavní příčinou výjimečných stavů bývá chyba programátora, která obvykle vede k tomu, že proměnná nebo parametr funkce má nečekanou hodnotu.

Pokud se program dostane do výjimečného stavu, je nutné, aby okamžitě přerušil výpočet a výjimečný stav pokud možno vyřešil. Vyšší programovací jazyky mají za tím účelem vytvořený více nebo méně sofistikovaný **systém výjimek**. Náš jazyk (jak už víme) prostě přeruší výpočet:

```
> (/ 1 0)
Error in function /: Division by zero with argument 1.
```

Pokud to programátor potřebuje udělat sám, má k dispozici funkci `error`. Vyhodnocením výrazu

```
(error fi1 ... fin)
```

dojde k vyvolání chyby a program přeruší práci. Chybová hláška je vytvořena z argumentů *fi1* ... *fin* stejně jako funkcí `out`.

Příklad bezpečné funkce na výpočet faktoriálu a jejího použití:

```
(defun factorial (n)
  (labels ((fact (n)
             (if (= n 0)
                 1
               (* n (fact (- n 1))))))
    (if (and (integerp n)
             (>= n 0))
        (fact n)
      (error "Argument " n " is not a non-negative integer. "))))
```

```
> (factorial 5)
120

> (factorial -1)
Error in function ERROR: Argument -1 is not a non-negative integer.
```

Stejně jako v minulém semestru i teď v tomto předmětu platí, že u všech funkcí, které budeme psát, se o správnost argumentů má starat uživatel. Detekce výjimečného stavu není povinnost a kvůli přehlednosti a soustředění na podstatu problému je obvykle lepší se o ni nestarat.

Pokud bychom ale výjimečný stav detekovat a ošetřit chtěli, musíme respektovat toto pravidlo:

Detekce výjimečného stavu

V jazycích používajících systém výjimek je vždy třeba po detekci výjimečného stavu vyvolat chybu funkcí ekvivalentní funkci `error`.

Druhá, zastaralá a nevhodná možnost, jejíž používání je bohužel stále rozšířeno, je nepřerušovat výpočet a vrátit z funkce nějakou chybovou hodnotu. Tento způsob je přijatelný pouze při programování na nižší úrovni, např. v jazyce C.

6 Funkce

V této části nejprve stručně zopakujeme základní informace o funkcích z minulého semestru a pak si povíme něco o obecnější podobě jejich λ-seznamů (dříve: seznamů parametrů).

Známe dva typy funkcí:

- vestavěné (primitiva)
- uživatelské (uživatelsky definované)

Uživatelské funkce obsahují

- seznam parametrů neboli λ-*seznam*
- tělo
- prostředí (tzv. **prostředí vzniku**)
- nepovinně dokumentaci

λ-seznam: seznam symbolů zvaných **parametry funkce**, před posledním může být klíčové slovo `&rest`. (Za chvíli rozšíříme o další možnosti.)

Tělo: seznam výrazů.

Prostředí: prostředí vzniku. Proto jsou funkce v Lispu **lexikální uzávěry**.

Dokumentace: textový řetězec.

Při aplikaci uživatelské funkce se vykonají následující kroky:

1. Vytvoří se prostředí, ve kterém se argumenty aplikace navážou na parametry funkce (podrobnosti dále).
2. Předkem prostředí se učiní prostředí vzniku funkce.

3. V novém prostředí se postupně (od prvního po poslední) vyhodnotí výrazy těla, hodnota posledního je návratovou hodnotou funkce.

Nové funkce lze vytvořit pomocí maker `lambda`, `defun`, `labels`:

`lambda` novou funkci vrací jako výsledek. Prostředí vzniku je aktuální prostředí.

`defun` funkci vytvoří a zadanému symbolu nastaví globální funkční vazbu na ni. Prostředí vzniku je aktuální prostředí (obvykle globální).

`labels` nová funkce je tzv. **lokální**. Prostředí vzniku i prostředí, ve kterém je vytvořena nová funkční vazba, je nové prostředí, jehož předkem je aktuální prostředí.

Dokumentace funkce

V definici funkce ji uvádíme jako řetězec za λ-seznam:

```
(defun add-zero (x)
  "Přičítá nulu k argumentu."
  (+ x 0))
```

λ-seznamy funkcí

Základní klíčová slova uvozující parametry daného typu:

`&optional` nepovinné parametry dané pozičně

`&rest` parametr pro seznam zbylých argumentů

`&key` nepovinné parametry dané názvem

Povinné pořadí při použití více klíčových slov: `&optional`, `&rest`, `&key`. Více klíčových slov v λ-seznamu současně ale obvykle nebudeme používat.

Parametry před klíčovými slovy jsou **povinné**. Nejjednodušší λ-seznam je seznam povinných parametrů. S tím jsme se setkávali většinu minulého semestru.

Klíčové slovo `&optional`

uvádí nepovinné parametry dané pozičně. Defaultní hodnota parametrů je `nil`. To lze změnit, pokud místo parametru uvedeme dvouprvkový seznam, jehož prvním prvkem je parametr a druhým výraz, který se v momentě aplikace funkce vyhodnotí na defaultní hodnotu.

`&optional`-parametry jsou navazovány na argumenty, dokud jsou argumenty k dispozici. Pak se použije defaultní hodnota (defaultní defaultní hodnota je `nil`).

λ-seznam	funkce aplikovatelná na
`(a b)`	dva argumenty
`(a b &optional c)`	dva nebo tři argumenty default třetího je `nil`
`(a b &optional c d)`	dva až čtyři argumenty default třetího a čtvrtého je `nil`
`(a b &optional (c 0) (d c))`	dva až čtyři argumenty default třetího je `0`, čtvrtého hodnota parametru `c`

Klíčové slovo `&rest`

uvádí parametr, který bude při aplikaci funkce obsahovat seznam všech zbylých argumentů. Umožňuje tedy definovat funkce akceptující neomezený (teoreticky) počet argumentů. Naváže se na seznam všech argumentů nepoužitých povinnými a `&optional`-parametry (tedy i argumentů pro `&key`-parametry; vyzkoušejte si to).

Klíčové slovo `&key`

uvádí nepovinné parametry dané jménem. Jako jména slouží symboly začínající dvojtečkou. Jsou to tzv. **klíče** a vyhodnocují se samy na sebe.

```
> :a
:A

> :key
:KEY
```

Parametry za klíčovým slovem `&key` jsou navazovány na argumenty podle **jména**. To se zadává při aplikaci pomocí klíčů. Ve funkci se definují a používají stejně jako `&optional`-parametry.

λ-seznam	seznam argumentů	a	x	y	z
`(a &key x y z)`	`(1 :x 2)`	1	2	nil	nil
`(a &key x y z)`	`(1 :y 3)`	1	nil	3	nil
`(a &key x y z)`	`(1 :x 2 :z 4)`	1	2	nil	4
`(a &key x (y 0) (z 0))`	`(1 :x 2 :z 4)`	1	2	0	4

Ve zdrojovém kódu k této přednášce je napsáno několik funkcí na práci se seznamy s využitím vhodných `&key`-parametrů. Většinu z nich jsme v jednodušší podobě programovali minulý semestr.

Symboly

Seznam obsahuje symboly definované v jazyce v minulém semestru, které budeme potřebovat, a nové symboly z této přednášky a cvičení.

Vestavěné z minulého semestru

globální proměnné (konstantní): `t`, `nil`, `pi`, `e`

proměnné listeneru: `*`, `**`, `***`, `+`, `++`, `+++`

speciální operátory: `if`, `bind`, `defun`, `labels`, `lambda`, `quote`, `function`

funkce: `eql`, `numberp`, `integerp`, `=`, `<`, `+`, `-`, `*`, `/`, `round`, `floor`, `sqrt`, `expt`, `log`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `functionp`, `funcall`, `apply`, `cons`, `consp`, `car`, `cdr`, `symbolp`

Uživatelské z minulého semestru

Jsou znovu definovány v jednom ze zdrojových souborů k této přednášce.

funkce: `not`, `>`, `>=`, `<=`, `1+`, `1-`, `plusp`, `zerop`, `minusp`, `abs`, `div`, `rem`, `evenp`, `oddp`, `gcd`, `min`, `max`, `identity`, `null`, `atom`, `caar`, `cadr`, `...`, `cddddr`, `length`, `nthcdr`, `nth`, `first`, `second`, `...`, `tenth`, `last`, `copy-list`, `list`, `revappend`, `reverse`, `tailp`, `ldiff`, `every`, `some`, `mapcar`, `mapcan`, `foldr`, `foldl`, `append`, `constantly`, `complement`

Vestavěné na této přednášce

globální proměnné (konstantní): všechny klíče

speciální operátory: `progn`, `setq`

funkce: `arrayp`, `aref`, `array-dimension`, `stringp`, `strs`, `stra`, `out`, `error`,

Uživatelské na této přednášce

funkce: `member`, `member-if`, `position`, `position-if`, `remove`, `remove-if`, `subsetp`, `union`, `set-difference`, `sort`

Uživatelské na cvičení

funkce: `find`, `find-if`, `count`, `count-if`, `mismatch`, `same-lists-p`, `intersection`, `remove-duplicates`

Otázky a úkoly na cvičení

Vylepšete další funkce z minulého semestru na práci se seznamy přidáním vhodných `&key`-parametrů:

1. Funkce `find`, `find-if`: parametry stejné jako u ukázkových funkcí `member` a `member-if` (tyto funkce můžete s výhodou použít), navíc parametr `:from-end`. *(do jazyka)*

2. Funkce `count-if`: parametr `:key`, `count`: parametry `:key` a `:test`. *(do jazyka)*

3. `mismatch`: parametry `:key`, `:test` a `:from-end`. V případě, že je `:from-end` *Pravda*, index se počítá vzhledem k prvnímu seznamu. *(do jazyka)*

4. `same-lists-p` pro dva argumenty: parametry `:key` a `:test`. Lze použít funkci `mismatch`. *(do jazyka)*

5. `intersection`: parametry `:key` a `:test`. *(do jazyka)*

6. `remove-duplicates`: parametry `:key`, `:test` a `:from-end`. Bez nastaveného `:from-end` by funkce měla vypustit vždy dřívější ze dvou prvků, které projdou testem. Je-li nastaveno na *Pravdu*, pak pozdější. *(do jazyka)*